



**SWAMINARAYAN
UNIVERSITY**
THE RESERVOIR OF ALL KNOWLEDGE

Faculty of Engineering (Degree)
Semester – 7th

Subject Name – Information security

Subject Code – BE103707

Faculty Name – Radhika Gokani

CHAPTER -3

1. Multiple encryption and triple DES

Multiple encryption refers to applying an encryption algorithm more than once—either with the same key or different keys—to strengthen security.

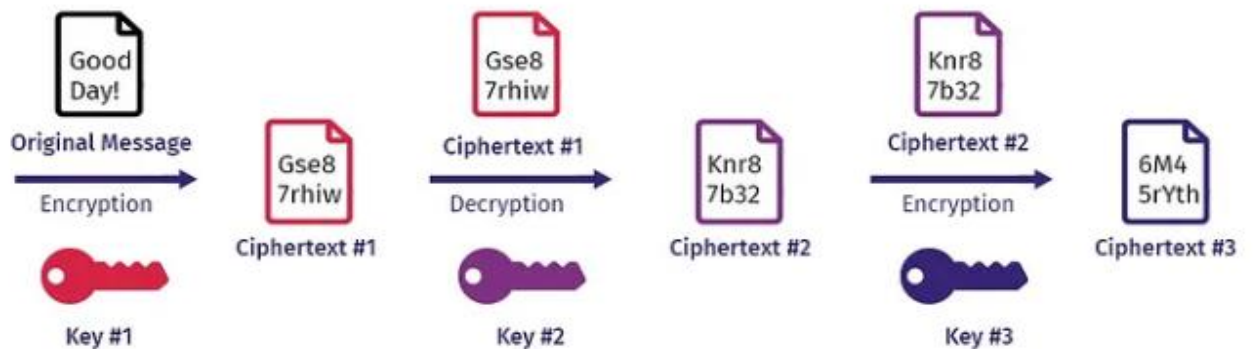
Double Encryption: Encrypts data twice using two different keys (K1 and K2).

- Formula: $C = E(K2, E(K1, P))$
- Decryption: $P = D(K1, D(K2, C))$

Triple Encryption: Encrypts data three times, often with three different keys.

- This is where **Triple DES** comes in.

How Triple DES (TDEA) Works



Triple DES (3DES) is an enhancement of the original DES (Data Encryption Standard), which became vulnerable to brute-force attacks due to its short key length (56 bits).

Triple DES applies the DES algorithm three times in sequence:

1. **Encrypt** with Key 1 (K1)
2. **Decrypt** with Key 2 (K2)
3. **Encrypt** again with Key 3 (K3)

This is known as the **Encrypt-Decrypt-Encrypt (EDE)** method:

- Formula: $C = E(K3, D(K2, E(K1, P)))$

Key Variants:

- **Two-Key 3DES:** Uses K1 and K2, with K1 reused for the third step.
- **Three-Key 3DES:** Uses three distinct keys (K1, K2, K3) for maximum security.

Why Triple DES?

- **Improved Security:** Triple DES increases the effective key length to 112 or 168 bits, making brute-force attacks far less feasible.
- **Backward Compatibility:** It allows systems using DES to upgrade without complete overhaul.
- **Legacy Use:** Still used in financial systems and legacy applications, though it's being phased out in favor of AES.

Limitations

- **Performance:** Slower than newer algorithms like AES.
- **Security Margin:** While stronger than DES, 3DES is considered outdated and vulnerable to certain attacks (e.g., meet-in-the-middle).

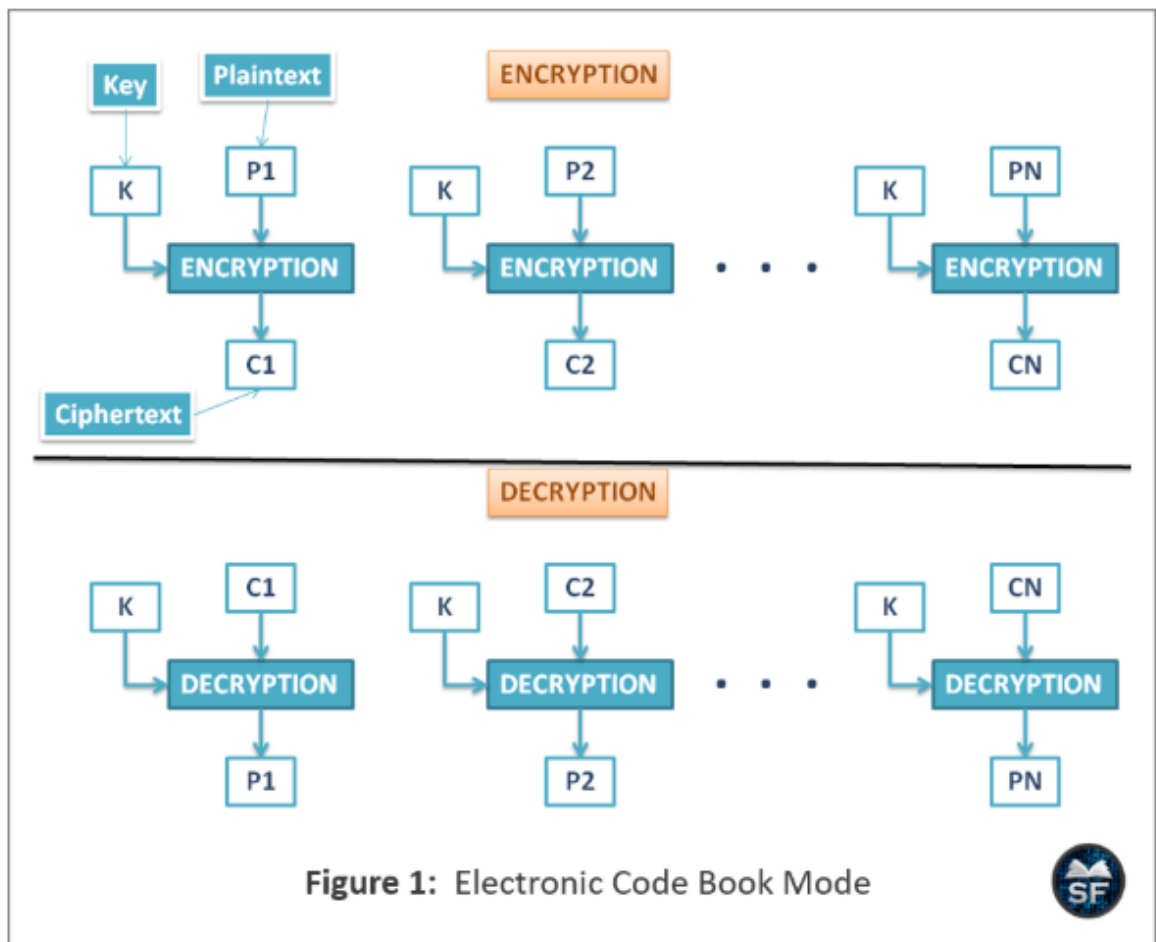
2. Electronic Code Book

Electronic Code Book (ECB) Mode in Cryptography

Electronic Code Book (ECB) is the **simplest mode of operation** for block ciphers like DES or AES. It's easy to implement but comes with serious security limitations.

ECB Works

- The plaintext is divided into fixed-size blocks (e.g., 64 or 128 bits).
- Each block is encrypted **independently** using the same key.
- Identical plaintext blocks produce **identical cipher text blocks**.



Example: If you encrypt the word "HELLOHELLO" using ECB, the two "HELLO" blocks will result in the same cipher text—revealing patterns.

Advantages of ECB

- **Simplicity:** Easy to implement and understand.
- **Parallel Processing:** Each block can be encrypted or decrypted independently.
- **Fast:** No chaining or feedback mechanisms slow it down.

Disadvantages of ECB

- **Pattern Leakage:** Repeated plaintext blocks show up as repeated ciphertext blocks.
- **No Diffusion:** Changes in one block don't affect others.

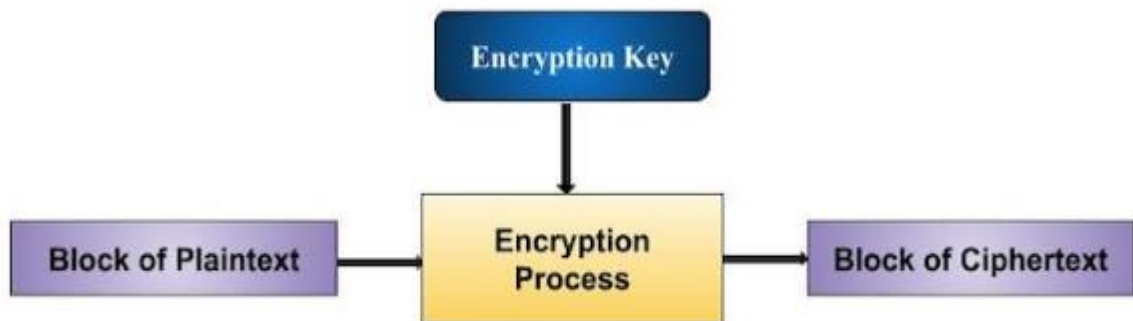
3. Cipher Block

Cipher Block in Cryptography

A **cipher block** refers to a fixed-size chunk of data that is processed by a **block cipher algorithm** during encryption or decryption. Block ciphers are foundational to modern cryptography and are used to secure data in applications like SSL/TLS, VPNs, and disk encryption.

What Is a Cipher Block?

- A **cipher block** is the **output** of a block cipher after encrypting a block of plaintext.
- It typically has a fixed size—**64 bits** (e.g., DES) or **128 bits** (e.g., AES).
- Each block is encrypted using a **symmetric key**, meaning the same key is used for both encryption and decryption.



How Block Ciphers Work

1. **Plaintext** is divided into equal-sized blocks.
2. Each block is processed through the cipher algorithm using a key.
3. The result is a **cipher block**—a scrambled, unreadable version of the original data.
4. Decryption reverses the process using the same key.

Common Block Cipher Algorithms

Algorithm	Block Size	Key Size	Status
DES	64 bits	56 bits	Obsolete
AES	128 bits	128/192/256 bits	Widely used
IDEA	64 bits	128 bits	Strong but less common
Twofish	128 bits	Variable	AES finalist

Algorithm	Block Size	Key Size	Status
Serpent	128 bits	128/192/256 bits	High security margin

Modes of Operation

Block ciphers use **modes of operation** to handle data larger than a single block:

- **ECB (Electronic Code Book):** Encrypts each block independently (not recommended for large data).
- **CBC (Cipher Block Chaining):** Each block depends on the previous cipher block.
- **CFB/OFB/CTR:** Introduce feedback or counters for better security and flexibility.

4. Chaining Mode

Chaining Mode in Cryptography

Chaining modes are techniques used with **block ciphers** to securely encrypt data larger than a single block. Instead of encrypting each block independently (like in ECB), chaining modes **link blocks together**, so the encryption of one block depends on the previous one—adding complexity and improving security.

Common Chaining Modes

Here are the most widely used chaining modes and how they work:

1. Cipher Block Chaining (CBC)

- Each plaintext block is **XORed** with the previous ciphertext block before encryption.
- The first block uses an **Initialization Vector (IV)** instead of a previous ciphertext.
- Formula: $C_n = E(K, P_n \oplus C_{n-1})$
- **Pros:** Hides patterns, good for file encryption.
- **Cons:** Not parallelizable, error propagation.

2. Cipher Feedback (CFB)

- Turns a block cipher into a **stream cipher**.
- Encrypts the previous ciphertext (or IV), then XORs with plaintext.
- **Pros:** Can encrypt data of any size.
- **Cons:** Slower than CTR, error propagation.

3. Output Feedback (OFB)

- Encrypts the IV, then uses the output to XOR with plaintext.
- The output is fed back into the cipher for the next block.
- **Pros:** No error propagation, good for noisy channels.
- **Cons:** Vulnerable if IV is reused.

4. Counter Mode (CTR)

- Encrypts a **counter value** (incremented for each block), then XORs with plaintext.
- **Pros:** Fully parallelizable, fast, secure.
- **Cons:** Requires careful counter management.

Why Use Chaining?

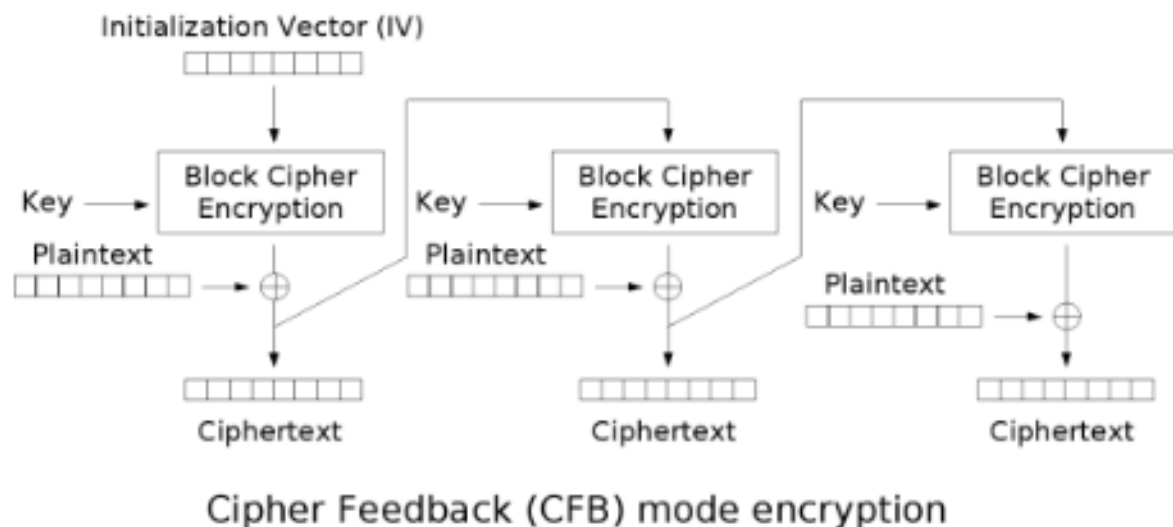
- **Security:** Prevents identical plaintext blocks from producing identical ciphertext.
- **Integrity:** Some modes (like CBC-MAC) can also verify data authenticity.
- **Flexibility:** Supports encryption of data streams and files of arbitrary length.

5. Cipher Feedback mode

Cipher Feedback (CFB) Mode is a block cipher mode of operation that cleverly transforms a block cipher into a stream cipher, allowing encryption of data in smaller units like bits or bytes rather than full blocks.

How It Works

- **Initialization Vector (IV):** Starts with a random IV loaded into a shift register.
- **Encryption Process:**
 1. The IV is encrypted using the block cipher and key.
 2. The most significant bits of the output are XORed with the plaintext segment to produce ciphertext.
 3. This ciphertext is fed back into the shift register for the next round.
- **Decryption:** Uses the same encryption function (not decryption!) to reverse the process.



Formula

- **Encryption:** $C_i = E_K(C_{i-1}) \oplus P_i$
- **Decryption:** $P_i = E_K(C_{i-1}) \oplus C_i$
 - Where E and K are the encryption function with key K , and C_0 is the IV.

Advantages

- Converts block cipher into a stream cipher—great for real-time data.
- Can encrypt data of arbitrary length.
- Resistant to certain types of cryptanalysis due to feedback chaining.

Limitations

- Error propagation: a single bit error in ciphertext affects multiple blocks.
- Not suitable for parallel encryption since each block depends on the previous one.

Common Use Cases

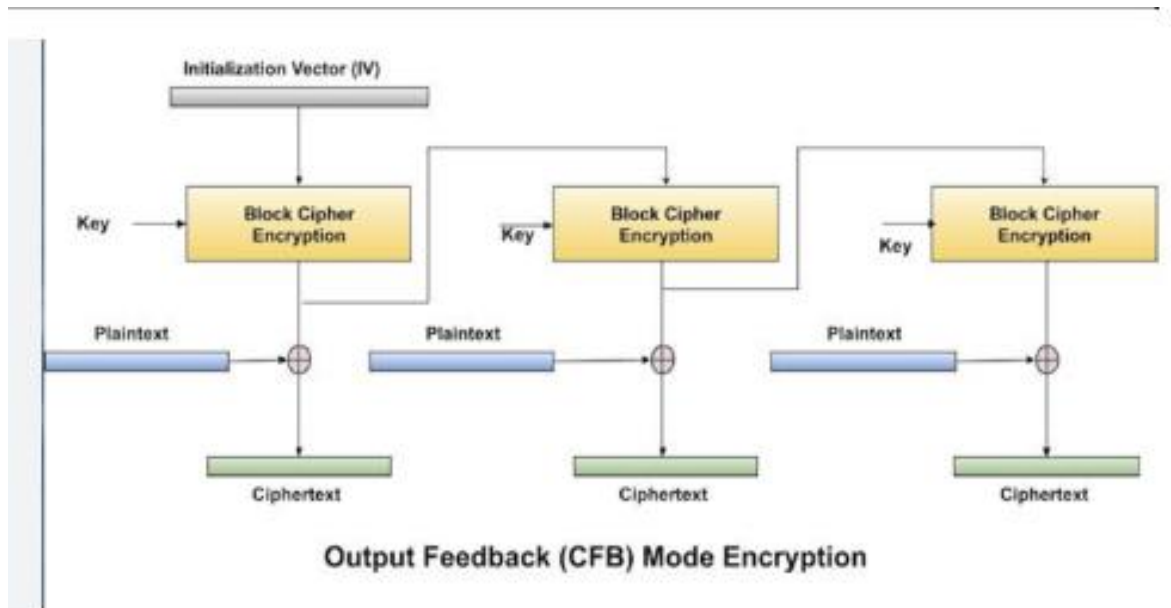
- Secure communication channels
- Antivirus software updates
- Data protection in streaming applications

6. Output Feedback mode

Output Feedback (OFB) Mode is a block cipher mode of operation that transforms a block cipher into a synchronous stream cipher—ideal for encrypting data in real-time or over noisy channels.

How It Works

- **Initialization Vector (IV):** A unique, random IV is used to kick off the encryption process.



- **Keystream Generation:**
 1. The IV is encrypted using the block cipher and key.
 2. The output is fed back into the cipher to generate the next keystream block.
 3. This process continues independently of the plaintext.
- **Encryption/Decryption:**
 - Plaintext is XORed with the keystream to produce ciphertext.
 - Decryption is identical—XOR the ciphertext with the same keystream.

Formula

- **Keystream:** $V_i = E_K(V_{i-1})$ $V_i = E_K(V_{i-1})$
- **Encryption:** $C_i = P_i \oplus V_i$ $C_i = P_i \oplus V_i$
- **Decryption:** $P_i = C_i \oplus V_i$ $P_i = C_i \oplus V_i$

Advantages

- **No error propagation:** A bit error in transmission affects only the corresponding bit in plaintext.
- **Parallelizable decryption:** Each block can be decrypted independently.
- **Stream cipher behavior:** Suitable for continuous or real-time data.

Limitations

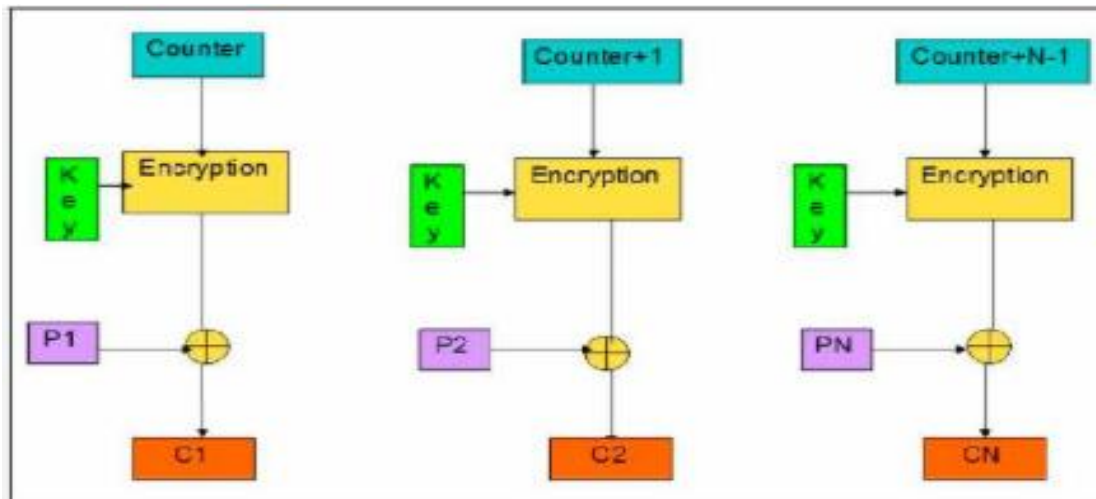
- **Keystream reuse risk:** Reusing the same IV with the same key compromises security.
- **No integrity check:** OFB doesn't detect tampering or message modification.
- **Vulnerable to certain attacks:** If IVs aren't truly random or unique, attackers can exploit patterns.

Use Cases

- Satellite communications
- Secure voice transmission
- Encrypted file systems with error tolerance

7. Counter mode

Counter (CTR) Mode is a block cipher mode of operation that transforms a block cipher into a stream cipher using a simple counter mechanism. It's widely appreciated for its speed, parallelizability, and simplicity.

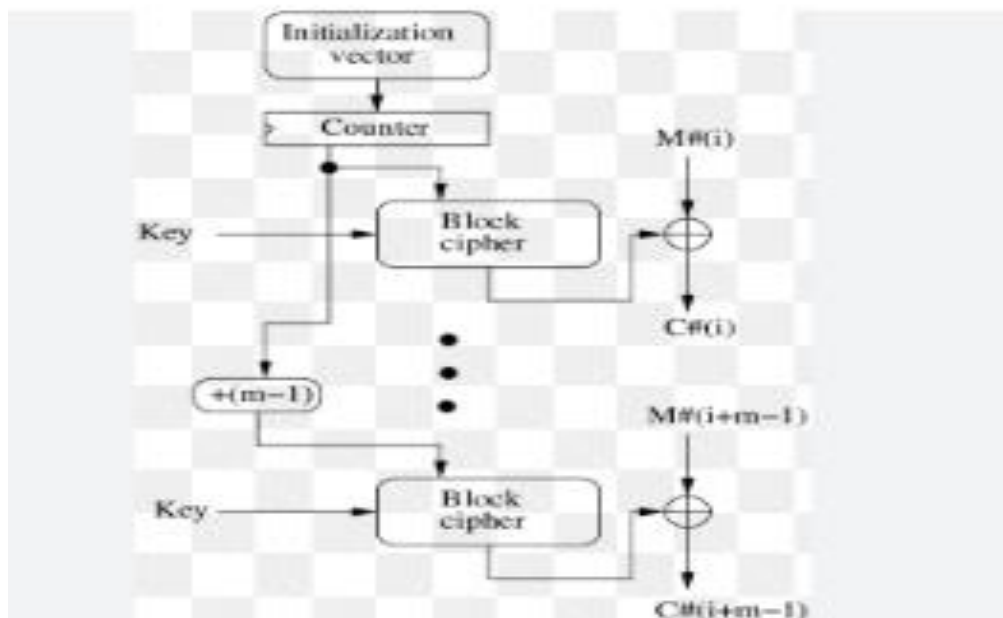


How It Works

- **Initialization:** Begins with a unique counter value (similar to an IV).
- **Keystream Generation:**
 1. The counter value is encrypted using the block cipher and key.
 2. The resulting output becomes the keystream block.
 3. This keystream is XORed with the plaintext to produce ciphertext.
 4. The counter is incremented for the next block.
- **Decryption:** Identical to encryption—XOR the ciphertext with the same keystream.

Formula

- **Keystream Block:** $V_i = E_K(\text{Counter}_i)$
- **Encryption:** $C_i = P_i \oplus V_i$
- **Decryption:** $P_i = C_i \oplus V_i$



Advantages

- **Parallel Processing:** Each block is independent, allowing simultaneous encryption/decryption.
- **No Error Propagation:** Bit errors affect only the corresponding block.
- **Preprocessing Friendly:** Keystream can be generated ahead of time.

- **Random Access:** You can decrypt any block without needing previous ones.
- **Simplicity:** Only the encryption function is needed—even for decryption.

Limitations

- **Counter Synchronization:** Sender and receiver must stay in sync.
- **Keystream Reuse Risk:** Reusing the same counter and key combination compromises security.
- **No Integrity Check:** CTR doesn't detect tampering or message modification.

Use Cases

- High-speed network encryption (e.g., VPNs)
- Disk encryption systems
- Secure real-time communications

CTR mode is often paired with authentication mechanisms like GCM (Galois/Counter Mode) to add integrity protection.
